

Technical Blog

In October 2021, Apache HTTP Server pushed out an emergency fix for a vulnerability that turned out to be one of the more embarrassing bugs in the project's recent history — not because it was complicated, but because it wasn't. CVE-2021-41773, and the incomplete patch that followed it, CVE-2021-42013, let an attacker read files off the server's filesystem with nothing more than a specially formatted URL. No login, no user interaction, just a single HTTP request. On servers where `mod_cgi` was also enabled, the same flaw could be pushed further into full remote code execution. Both were rated 9.8 out of 10 on the CVSS scale, and within hours of the advisory going public, mass scanning for vulnerable servers was already underway.

What is the vulnerability

At its core, this is a path traversal bug. Apache is supposed to keep requests confined to the document root you've configured — you ask for `/images/logo.png`, and it serves exactly that file and nothing outside that directory. The vulnerability broke that boundary. By encoding a single dot in a URL as `%2e` instead of a literal period, an attacker could smuggle a `"../"` sequence past Apache's directory checks and walk the request path straight out to the filesystem root, then back down into files like `/etc/passwd`.

Where it got worse was CGI. If a server had `mod_cgi` loaded — common on older shared-hosting setups — the same escape technique could route a request into `/bin/sh` instead of a file. Whatever was sent in the POST body got executed as a shell command. That's the difference between someone reading your password file and someone running arbitrary commands on your box.

Why it happens

The bug lives in a function called `ap_normalize_path()`, inside `server/util.c`, whose entire job is to take a raw request path and collapse it down to something safe — turning `/a/b/../c` into `/a/c`, and rejecting anything that tries to climb above the allowed directory. In 2.4.49, the order of operations inside that function got mixed up. Apache checked the path for traversal sequences first, and only afterward decoded percent-encoded characters. So a segment written as `..%2e` passed the traversal check looking harmless, got decoded into `..` after the fact, and Apache never went back to re-check it. The validation had already happened on the "before" version of the string, not the "after."

Apache's own fix in 2.4.50 tried to close this by explicitly rejecting any path containing `%2e`. It didn't hold. The check only decoded once, and attackers found that double-encoding the percent sign itself — writing `%25` as `%%32%65` — slipped through the single-pass filter, then unfolded into `..` once Apache finished processing the request. That's CVE-2021-42013. The real fix, in 2.4.51, finally decoded the path fully before doing any traversal checks at all, rather than trying to

pattern-match specific encodings.

One thing worth being clear about: this bug alone doesn't hand you the keys. Apache's default configuration denies access to the filesystem root, so a fresh install isn't exploitable even with the flawed function underneath. It needs a second ingredient — a server where an administrator has set `Require all granted` on the root directory, usually to fix some unrelated permissions headache. That combination wasn't rare in 2021.

How I reproduced it

I built this out as an isolated Docker lab rather than touching anything internet-facing — two containers, one running the actual vulnerable `httpd:2.4.49` image pulled straight from Docker Hub, configured with `mod_cgi` on and the permissive directory setting described above, and a second running `httpd:2.4.51` with hardened defaults, as a before-and-after comparison. Both bind only to `127.0.0.1`, so nothing leaves the machine.

Against the vulnerable container, a request like `GET /icons/.%2e/%2e%2e/%2e%2e/etc/passwd` comes back with a 200 and the contents of `/etc/passwd` in the response body — file disclosure achieved. Chaining it into the CGI handler, a `POST` to `/cgi-bin/.%2e/.%2e/bin/sh` with a body of `echo; id` gets you back the output of the `id` command, confirming code execution rather than just a file read. Running the exact same requests against the patched 2.4.51 container gets a flat 400 Bad Request both times — the fixed version rejects the malformed path before it ever reaches the point of resolving to a real file.

Detecting it

Detection here works on two levels. The lazy version is a banner check — Apache happily reports its own version number in the `Server` response header unless you've told it not to, so anything reporting 2.4.49 or 2.4.50 is worth flagging immediately. But that's not reliable on its own, since plenty of admins turn on `ServerTokens Prod` specifically to hide the version string, which would make a genuinely vulnerable box look clean on a banner check alone.

The more trustworthy method is an active, non-destructive probe: send the traversal pattern at a harmless file — something like `/etc/issue` instead of `/etc/passwd` — and just watch whether you get a 200 (traversal succeeded, server is vulnerable) or a 400 (server rejected the malformed path, server is patched). That's the approach I used in the scanning script for this lab, and it doesn't depend on the server being honest about its own version.

Fixing it

There's really one correct fix: upgrade to Apache 2.4.51 or later. That's the only version where the underlying decode-then-validate ordering was actually corrected, rather than patched around with pattern matching. Everything else is a mitigation, not a cure. If you can't upgrade immediately, setting `Require all denied` explicitly on `<Directory />` removes the misconfiguration that makes the traversal exploitable in the first place, and disabling `mod_cgi` where it isn't needed closes off the RCE path even if traversal is still technically possible. Turning on `ServerTokens Prod` won't stop an attacker who's actually probing your server, but it does deny easy reconnaissance to anyone just scanning banners across the internet looking for easy targets.

Key takeaways

What stands out most about this CVE, looking back at it, isn't the traversal bug itself — path traversal bugs are old news, and Apache has had similar issues before. It's how a single reordering of two operations inside one function created a critical, unauthenticated RCE in one of the most widely deployed pieces of server software on the planet, and how the first attempt at a fix still wasn't enough. It's a good reminder that patching by blocking a specific known payload pattern, rather than fixing the actual logical flaw underneath it, tends to just move the goalposts for whoever's looking for the next bypass. The people who found CVE-2021-42013 didn't need to find a new bug — they just needed to double-encode the old one.